Group Members Name:

Kashaf Sajjad (F2024105071)

Meral Azeem (F2024105088)

Zoya (F2024105112)

Saira Iqbal (F2024105316)

Section:Y3

Course: Open-Source Software development

Submitted to: Sir Ali Harris

OSSD Project Report

School of System and Technology

University of Management and Technology

Table of Contents

Part A Library Overview

Name of the Library

The library chosen for this project is Pandas, one of the most widely used open-source Python libraries for working with data.

Purpose of the Library

Pandas makes it easy to load, clean, organize, and study data in Python. It gives Python two main tools: the Series (a single column of data) and the DataFrame (a full table with rows and columns, like a spreadsheet). With these tools, a programmer can read data, look at it, change it, and save it again, using simple commands.

Problems It Solves

Before pandas, working with tables of data in plain Python was slow and difficult. Programmers had to write a lot of extra code just to read a CSV file, search through rows, or calculate totals. Pandas solves these problems by giving ready-made tools for common tasks such as:

- Reading data from files like CSV, Excel, JSON, and SQL databases
- Cleaning messy data, such as filling in missing values or removing duplicate rows
- Filtering, sorting, and searching large tables of data quickly
- Grouping data and calculating totals, averages, and other statistics
- Joining or combining multiple tables of data together
- Working smoothly with other libraries such as NumPy, Matplotlib, and Scikit-learn

Major Features

- DataFrame and Series objects for storing and organizing labeled data
- Fast handling of missing data (shown as NaN, NA, or NaT)
- Tools for time series data, such as dates and time-based calculations
- Powerful grouping feature (groupby) for summarizing data
- Ability to merge, join, and reshape multiple datasets
- Reading and writing many file formats: CSV, Excel, JSON, SQL, Parquet, and more
- A new, dedicated string data type and Copy-on-Write behaviour introduced in the pandas 3.0 release for more predictable and faster code

License Used

Pandas uses the BSD 3-Clause License. This is an open license, so anyone can use, copy, change, or even sell software made with pandas, as long as they keep the original copyright note. This easy license is one reason pandas is used so widely, in both small projects and big companies.

Current Version

At the time this report was written (mid-2026), the most recent stable release of pandas was version 3.0.3, released on 11 May 2026. This came after the major version 3.0.0 release on 21 January 2026, which introduced big changes such as a dedicated string data type and Copy-onWrite as the default behaviour.

Year of Initial Release

Development of pandas began in 2008 at AQR Capital Management, a quantitative hedge fund, where it was created to make financial data analysis easier in Python. It was later released as open-source software and has been actively maintained by the community ever since.

Industries and Applications Where It Is Used

Because pandas makes data handling so much easier, it is used very widely across many fields, including:

- Finance and banking — analyzing stock prices, trading data, and risk models
- Data science and machine learning — cleaning and preparing data before training models
- Academic research — organizing and analyzing experiment or survey data
- Healthcare — studying patient records and medical research data
- Web analytics and marketing — analyzing user behaviour and campaign performance
- Government and public policy — processing census and statistical data

---

Part B GitHub Repository Analysis

Repository Information

The table below summarizes the key facts about the official pandas GitHub repository, as observed in mid-2026:

Repository URL	https://github.com/pandas-dev/pandas
Organization	pandas-dev
Number of Stars	Approximately 49,100 (49.1k)
Number of Forks	Approximately 20,000 (20k)
Open Issues	Approximately 2,877
Open Pull Requests	Approximately 266

Number of Contributors	Over 3,700 individual contributors across the project's history
Latest Release	Version 3.0.3, released 11 May 2026
License	BSD 3-Clause “New” or “Revised” License
Primary Language	Python (with some Cython and C for performance-critical code)

Visit the official pandas repository on GitHub

Repository Structure

The pandas repository is organized in a clean and standard way, which is common among large, mature open-source projects. The main folders and files include:

- pandas/ — the main source code folder, containing the actual logic for DataFrame, Series, and all data operations
- pandas/tests/ — thousands of automated tests that check the library works correctly after every change
- doc/ — the source files used to build the official documentation website
- scripts/ — helper scripts used by maintainers for tasks like checking code style
- ci/ — configuration files for Continuous Integration (automatic testing on GitHub)
- .github/ — GitHub-specific files, including issue templates, pull request templates, and contribution workflows
- README.md — the introductory file all visitors see first
- setup.py and pyproject.toml — files that describe how to build and install the package
- environment.yml — file listing the dependencies needed for development
- LICENSE — the file containing the full text of the BSD 3-Clause License

Purpose of the README

The README.md file is the first thing most visitors see when they open the repository. Its purpose is to quickly explain what the project does, why it is useful, and how to get started. The pandas README includes a short description of the library, badges showing build status and version, installation instructions, links to documentation, and information on how to contribute. A good README acts like a “front door” to the project, helping new users decide if the library is right for them and helping new contributors find their way in.

Purpose of Requirements / Setup Files

Files such as pyproject.toml, setup.py, and environment.yml tell Python and other tools exactly what is needed to install and run the project. They list the required dependencies (such as NumPy and python-dateutil), the minimum supported Python version, and instructions for building the package from source. These files make sure that anyone installing pandas, whether a regular user or a new contributor, gets a working setup with the correct versions of everything pandas depends on.

Community Activity

Pandas has a very active community, as shown by the following figures observed on the repository:

- Open Issues: approximately 2,877 ,these include bug reports, feature requests, and discussion threads
- Open Pull Requests: approximately 266 , these are contributions waiting for review by the core team
- Recent Commit Activity: the main branch receives commits on a near-daily basis, with regular patch releases (for example, version 3.0.0 in January 2026, followed by 3.0.1, 3.0.2, and 3.0.3 over the following months)

This level of activity shows that pandas is not an abandoned project. It is maintained by a dedicated core team and supported by thousands of outside contributors who report bugs, suggest features, and submit fixes.

Contribution Process

Pandas follows a standard GitHub “fork and pull request” workflow, which is common across most large open-source projects. The general steps a new contributor follows are:

1. Create a personal copy (a “fork”) of the pandas repository on GitHub
2. Clone that fork to their own computer using Git
3. Create a new branch for the specific change they want to make
4. Make the change, write or update tests, and follow the project's coding style
5. Push the branch back to their own fork on GitHub
6. Open a Pull Request (PR) asking the pandas maintainers to merge their change into the main project
7. Respond to feedback from reviewers and update the PR until it is approved and merged

Contribution Guidelines

Pandas maintains a detailed contributing guide that explains coding standards, how to set up a development environment, how to write and run tests, and how to format commit messages. New contributors are encouraged to start with issues labeled “good first issue” or “Docs,” which are chosen to be approachable for newcomers. The guide also explains an important and fairly modern policy: contributors are allowed to use AI coding assistants, but they must disclose this, fully review and understand any AI-generated code, and make sure it follows all of the project's normal contribution rules.

Code of Conduct

Pandas has an official Code of Conduct that all contributors and maintainers are expected to follow. It sets expectations for respectful, professional behaviour within the community, covering things like respectful communication, handling disagreements constructively, and how to report violations. Having a clear code of conduct helps keep the community welcoming, especially for new or inexperienced contributors.

Analysis: An Interesting Issue

Among the open issues in the repository is one related to indexing behaviour in DataFrames and Series — a recurring and technically tricky area of the codebase because indexing touches almost every other part of pandas. Issues in this category are usually opened by users who notice unexpected behaviour while filtering or selecting data, and they are tagged with labels such as “Bug” and “Indexing.”

Purpose: The issue exists to flag a case where the library's behaviour does not match what users expect, often due to an edge case in how rows or columns are selected.

Outcome: Once reported, such issues are typically reviewed by the core team, who decide whether it is a genuine bug, confirm it can be reproduced, and label it for further discussion or for a contributor to pick up and fix. Indexing issues often need careful discussion before a fix is accepted, since a small change in this part of the code can affect many other features.

Analysis: An Interesting Pull Request

On the pull request side, an interesting example is one of the routine maintenance PRs that update pandas' Continuous Integration (CI) and packaging setup — for instance, work related to publishing build wheels for new platforms, such as Windows ARM64 or free-threaded Python builds, alongside the 3.0 release cycle.

Purpose: These pull requests aim to keep pandas compatible with new hardware, new Python versions, and modern, secure publishing methods (such as PyPI Trusted Publishing).

Outcome: After being reviewed by maintainers for correctness and tested through the project's automated CI pipeline, such PRs are merged into the main branch and included in the next release. This kind of behind-the-scenes work is just as important as new features, because it keeps the library usable on the latest systems that real users actually have.

Part C Documentation Review

Installation Instructions

Pandas provides clear installation instructions on its official documentation site. Users can install it with a simple command, such as pip install pandas or conda install pandas, depending on which package manager they use. The documentation also explains the minimum required version of Python (3.11 or higher, for pandas 3.0) and lists optional libraries, like PyArrow, that improve performance.

Usage Examples

The official documentation includes a “Getting Started” section filled with short, beginner-friendly code examples. These show how to create a DataFrame, select data, and perform simple operations. The examples are written so that someone with basic Python knowledge can follow along without needing to understand the internal workings of the library first.

Tutorials

Beyond simple examples, pandas offers a structured set of tutorials, often called the “10 minutes to pandas” guide and a longer “User Guide.” These walk through common real-world tasks such as cleaning data, merging tables, handling missing values, and working with dates. The tutorials build on each other, moving from simple ideas to more advanced ones, which makes the learning curve smoother for newcomers.

API Documentation

Pandas has a very thorough API reference, which lists every public function, class, and method in the library, along with its parameters, return values, and small examples. This is extremely useful for experienced users who already know what they want to do and just need to check the exact name or options of a function.

Contribution Guidelines

As discussed in Part B, pandas also documents how to contribute to the project itself, not just how to use it. This documentation covers setting up a development environment, writing tests, and following the project's coding style, and is kept alongside the rest of the technical documentation.

Strengths of the Documentation

- Very thorough and detailed, covering almost every function in the library
- Organized into clear sections for different needs: getting started, tutorials, user guide, and API reference
- Many real, runnable code examples that users can copy and test immediately
- A dedicated “What's New” / release notes page that clearly lists changes for every version, including breaking changes

Weaknesses of the Documentation

- Because pandas has so many features, the documentation can feel overwhelming to complete beginners
- Some advanced topics, such as performance tuning or the newer Copy-on-Write behaviour, assume the reader already understands earlier pandas concepts well
- Search results on the documentation website can sometimes return too many similarlooking functions, making it harder to find the exact one needed quickly

Suggested Improvements

- Add a clearer, very short “absolute beginner” path for people who have never used a DataFrame before
- Include more visual diagrams showing how data moves and changes during merge, groupby, and reshape operations
- Provide more guidance on common error messages, helping new users understand and fix mistakes faster

Part D Open-Source Ecosystem Reflection

Why Is This Project Successful?

Pandas is successful for several connected reasons. First, it solves a real and very common problem: working with structured, table-like data in Python, which almost every data-related task needs in some way. Second, it integrates smoothly with other major tools in the Python data science ecosystem, such as NumPy, Matplotlib, and Scikit-learn, so learning pandas pays off across many other tools as well. Third, its permissive BSD license means companies can use it freely in commercial products without legal concerns, which has helped it spread into industry, not just academia. Finally, years of continuous, active maintenance and improvement (now reaching major version 3.0) have kept it fast, reliable, and up to date with modern Python versions.

How Does the Community Support the Project?

The pandas community supports the project in many ways beyond just writing code. Users report bugs and unexpected behaviour through GitHub issues, which helps maintainers find and fix problems they might not have noticed themselves. Experienced contributors review and test pull requests submitted by others, helping catch mistakes before they reach the main codebase. The project also holds regular community meetings, including ones specifically for new contributors, which helps bring in fresh volunteers over time. Beyond GitHub, communication channels such as a Slack workspace and mailing lists allow contributors to discuss design decisions and get help when they are stuck.

What Challenges Might Maintainers Face?

- Reviewing a very high number of incoming issues and pull requests with a limited number of core maintainers, which can create bottlenecks
- Balancing the needs of long-time users, who want stability, against the desire to add new features or fix design mistakes from years ago
- Making breaking changes carefully, since millions of existing programs and businesses depend on pandas working a certain way
- Keeping documentation, tests, and examples updated as the library keeps growing in size and complexity
- Avoiding contributor burnout, since most open-source maintenance work is done as volunteer effort alongside other jobs

How Important Are Contributors to the Project's Growth?

Contributors are essential to pandas' growth and survival. With thousands of contributors having participated over the years, no small team could have built or maintained a library this large and detailed alone. Outside contributors bring fresh use cases, catch bugs that maintainers might miss, write documentation, and sometimes bring entirely new feature ideas based on problems

they faced in their own work. The clear contribution guidelines, labeled “good first issue” tags, and regular new-contributor meetings all show that the project actively values and depends on this outside participation, rather than treating it as a side benefit.

What Did Your Group Learn About Open-Source Development?

Studying pandas helped us understand that open-source software is not simply “free code on the internet,” but a living project shaped by an organized community. We learned how licensing affects who can use and build on a project, how GitHub features like issues and pull requests structure collaboration between strangers across the world, and how documentation quality can make or break a tool's adoption. We also saw that even a hugely successful project like pandas still has thousands of open issues and ongoing debates, showing that open-source development is a continuous, evolving process rather than something that is ever fully “finished.”

Part E Optional Demonstration (Bonus)

Mini Example Program

As the bonus demonstration, we created a short Python script that shows basic, beginner-level usage of pandas. The script (pandas_demo.py) performs the following steps:

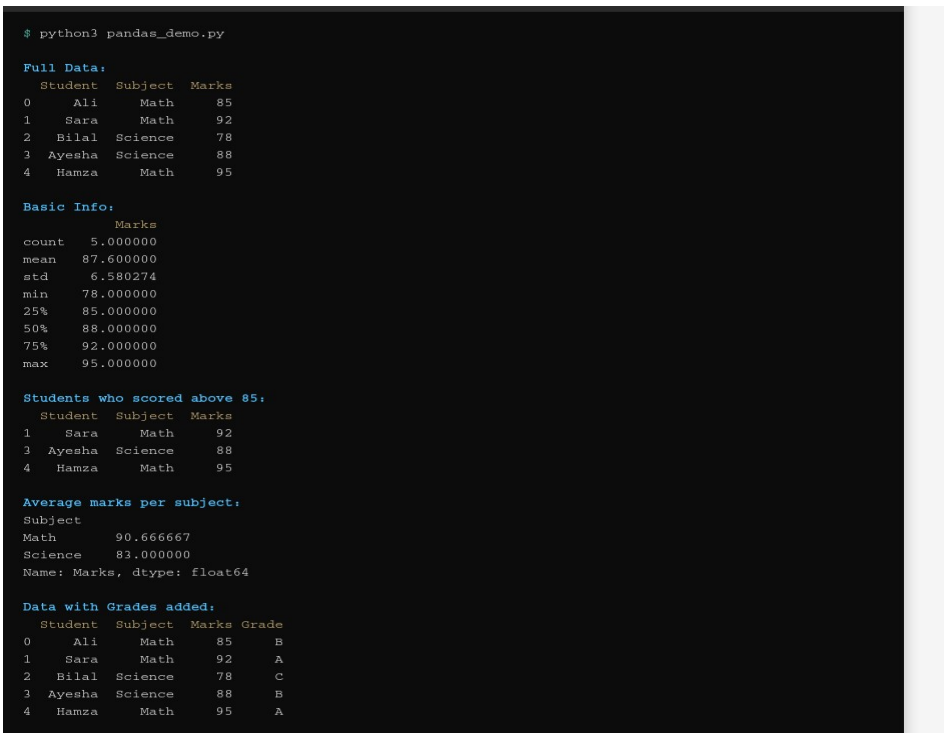
1. Creates a small DataFrame containing student names, subjects, and marks
2. Prints basic statistics about the data using describe()
3. Filters the data to find students who scored above a certain mark
4. Groups the data by subject and calculates the average mark per subject
5. Adds a new column that assigns a letter grade based on the marks
6. Saves the final table to a new CSV file named student_results.csv

This program was tested and runs successfully, producing correct, readable output at every step. It demonstrates, in a simple and beginner-friendly way, exactly the kind of data-handling tasks described in Part A: reading data into a table, filtering it, summarizing it with groupby, and exporting the result, which are the same core operations that make pandas useful across finance, research, and data science in the real world.

Screenshot of the Program Code

```python
# Mini Demonstration Program – Pandas Library
import pandas as pd

# 1. Create a simple DataFrame (like a small table of student marks)
data = {
    "Student": ["Ali", "Sara", "Bilal", "Ayesha", "Hamza"],
    "Subject": ["Math", "Math", "Science", "Science", "Math"],
    "Marks": [85, 92, 78, 88, 95],
}
df = pd.DataFrame(data)

print("Full Data:")
print(df)

# 2. Basic inspection
print(df.describe())

# 3. Filter rows: students who scored more than 85
top_students = df[df["Marks"] > 85]
print(top_students)

# 4. Group by Subject and find average marks
average_marks = df.groupby("Subject")["Marks"].mean()
print(average_marks)

# 5. Add a new column: Grade based on marks
def get_grade(marks):
    if marks >= 90:
        return "A"
    elif marks >= 80:
        return "B"
    else:
        return "C"

df["Grade"] = df["Marks"].apply(get_grade)
print(df)

# 6. Save the final result to a CSV file
df.to_csv("student_results.csv", index=False)
```

Screenshot of the Program Output



```
$ python3 pandas_demo.py

Full Data:
  Student  Subject  Marks
0     Ali     Math     85
1    Sara     Math     92
2   Bilal  Science     78
3  Ayesha  Science     88
4   Hamza     Math     95

Basic Info:
           Marks
count   5.000000
mean   87.600000
std     6.580274
min    78.000000
25%    85.000000
50%    88.000000
75%    92.000000
max    95.000000

Students who scored above 85:
  Student  Subject  Marks
1    Sara     Math     92
3  Ayesha  Science     88
4   Hamza     Math     95

Average marks per subject:
Subject
Math       90.666667
Science    83.000000
Name: Marks, dtype: float64

Data with Grades added:
  Student  Subject  Marks Grade
0     Ali     Math     85     B
1    Sara     Math     92     A
2   Bilal  Science     78     C
3  Ayesha  Science     88     B
4   Hamza     Math     95     A
```

References

Official pandas GitHub repository

Official pandas documentation

Pandas release notes / What's New

Pandas 3.0 announcement blog post

Pandas contributing guide

Pandas “good first issue” contribution page

Pandas open issues tracker